Assignment: Build an Adaptive RAG Inference System

Objective

Build a mini RAG pipeline that optimizes itself at inference time.

◆ Part 1: Basic Pipeline

Implement:

● Document ingestion

● Vector index (FAISS or similar)

● Query → retrieve → generate pipeline

◆ Part 2: Retrieval Optimization

Add:

● Dynamic top-K (not fixed)

● Hybrid retrieval:

○ vector search

○ keyword filtering

● Simple re-ranking

◆ Part 3: Adaptive Decision Layer

Implement a runtime selector:

Example:

● If query is short → smaller K

● If query is complex → larger K

● If latency high → reduce retrieval depth

This can be rule-based OR simple scoring logic

◆ Part 4: Feedback Loop (Important)

Add a simple feedback mechanism:

● Track:

○ latency

○ response quality proxy (e.g., answer length, confidence, or heuristic)

● Adjust:

○ top-K

○ retrieval strategy

No training required — just adaptive logic

🔹 **Part 5: Performance Measurement**

Report:

● Latency (P50 / P95)

● Retrieval time vs generation time

● Impact of adaptive logic

Deliverables

● GitHub repo

● README with:

○ architecture diagram

○ design decisions

○ tradeoffs

● Short report:

○ what worked / didn't

○ how system adapts

Bonus

● Query decomposition (multi-step queries)

● Caching layer

● ANN tuning experiments

● Model routing (small vs large model)